

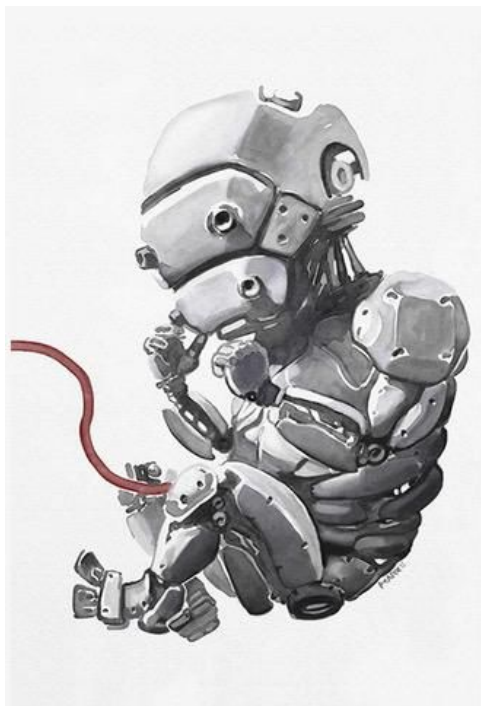
"For all those that have to fight for the respect that everyone else is given without question."

— N.K. Jemisin, The Fifth Season



“Low expectations are a self-fulfilling prophecy. If we aim high, we’ll get better results.”

— Ted Chiang, The Lifecycle of Software Objects



CMPUT 365

Introduction to RL

Plan

- Overview of Bandits, MDPs, and Dynamic Programming
 - It is not because I said something today that that thing will appear in the midterm. Likewise, it is not because I did not mention something that that thing won't be in the midterm.
- Do some exercises along the way. I will need to give you time to do them.
- Don't think about this class as being about the midterm, but as I situating you.
 - I really care about the **why**.
- I might not finish everything today, and it is fine. It is a review after all.

Reminder (1 of 2)

You **should be enrolled in the private session** we created in Coursera for CMPUT 365.

I **cannot** use marks from the public repository for your course marks.

You **need** to **check, every time**, if you are in the private session and if you are submitting quizzes and assignments to the private section.

The deadlines in the public session **do not align** with the deadlines in Coursera.

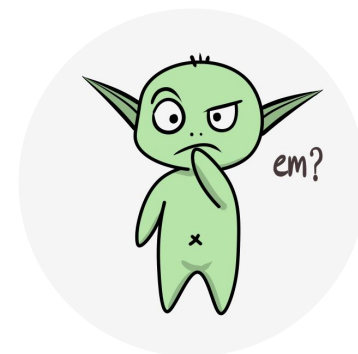
If you have any questions or concerns, **talk with the TAs** or email us
`cmput365@ualberta.ca`.

Reminder (2 of 2)

The midterm is Friday!

- It will be written here, in the classroom (50 minutes).
- I need you to bring some piece of ID.
- It'll have 6 multiple-choice (1.5 marks) and 3 open-ended questions (3, 4, 4 marks).
- There will be three versions of the same exam, since the classroom is so “cozy”.
- It is closed-book, no calculators are allowed.
- There won't be a formula sheet.
- We still have office hours today, twice tomorrow, and Friday.

Please, interrupt me at any time!



Reinforcement learning

Reinforcement learning is a computational approach to learning from interaction to maximize a numerical reward signal (Sutton & Barto; 2018)

- The idea of learning by interacting with our environment is very natural
- It is based on the idea of a learning system that wants something, and that adapts its behavior to get that



Some features are unique to reinforcement learning:

- Trial-and-error
- The trade-off between exploration and exploitation
- The delayed credit assignment / delayed reward problem

Bandits (Chapter 2) as a simplified setting/example

**MDPs (Ch. 3) give us a language,
Dynamic Programming (Ch. 4) give us the
first set of solutions (and intuition)**

The Agent-Environment Interface

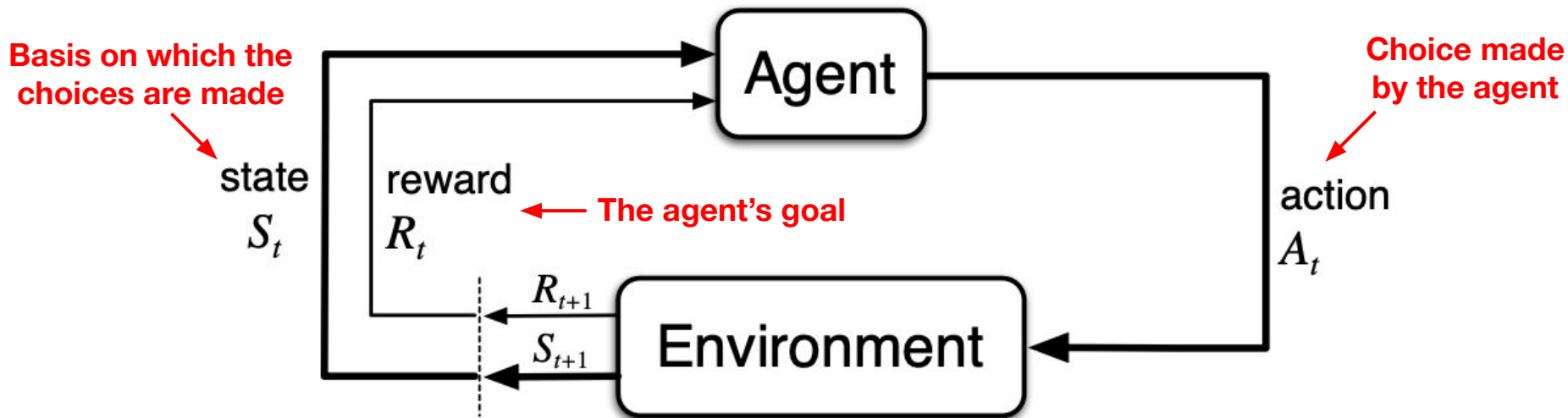
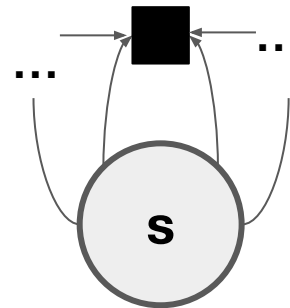


Figure 3.1: The agent–enviro

$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$

The bandits problem as an MDP

- In the MDP language:
 - There's a single state, and it doesn't matter what we do, we will always terminate after taking an action (thus, no delayed consequences)
 - We don't have access to $p(s', r | s, a)$, we don't know how the world works. Thus, we need to explore.
 - Note that, in the bandits problem, $p(s', r | s, a)$ effectively becomes $p(r | a)$.



Exercise: If we were to adjust the Bellman equations for q_π and q_* to the bandits case, what would they look like? Remember:

$$q_\pi(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') q_\pi(s', a') \right]$$

$$q_*(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right]$$



Updating estimates online and exploring (w/ ε -greedy)

$$Q_{n+1} = Q_n + \frac{1}{n} [R_n - Q_n]$$

NewEstimate $\leftarrow$ OldEstimate + StepSize [Target - OldEstimate]

A simple bandit algorithm

Initialize, for $a = 1$ to k :

$Q(a) \leftarrow 0$

$N(a) \leftarrow 0$

Loop forever:

$A \leftarrow \begin{cases} \operatorname{argmax}_a Q(a) & \text{with probability } 1 - \varepsilon \quad (\text{breaking ties randomly}) \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$

$R \leftarrow \text{bandit}(A)$

$N(A) \leftarrow N(A) + 1$

$Q(A) \leftarrow Q(A) + \frac{1}{N(A)} [R - Q(A)]$

Alternative to ε -greedy exploration

- Optimism in the face of uncertainty

A simple bandit algorithm

Initialize, for $a = 1$ to k :

$Q(a) \leftarrow 0$ ~~100~~ (a really large value)

$N(a) \leftarrow 0$

Loop forever:

$A \leftarrow \begin{cases} \arg\max_a Q(a) & \text{with probability } 1 - \varepsilon \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$ ~~(breaking ties randomly)~~

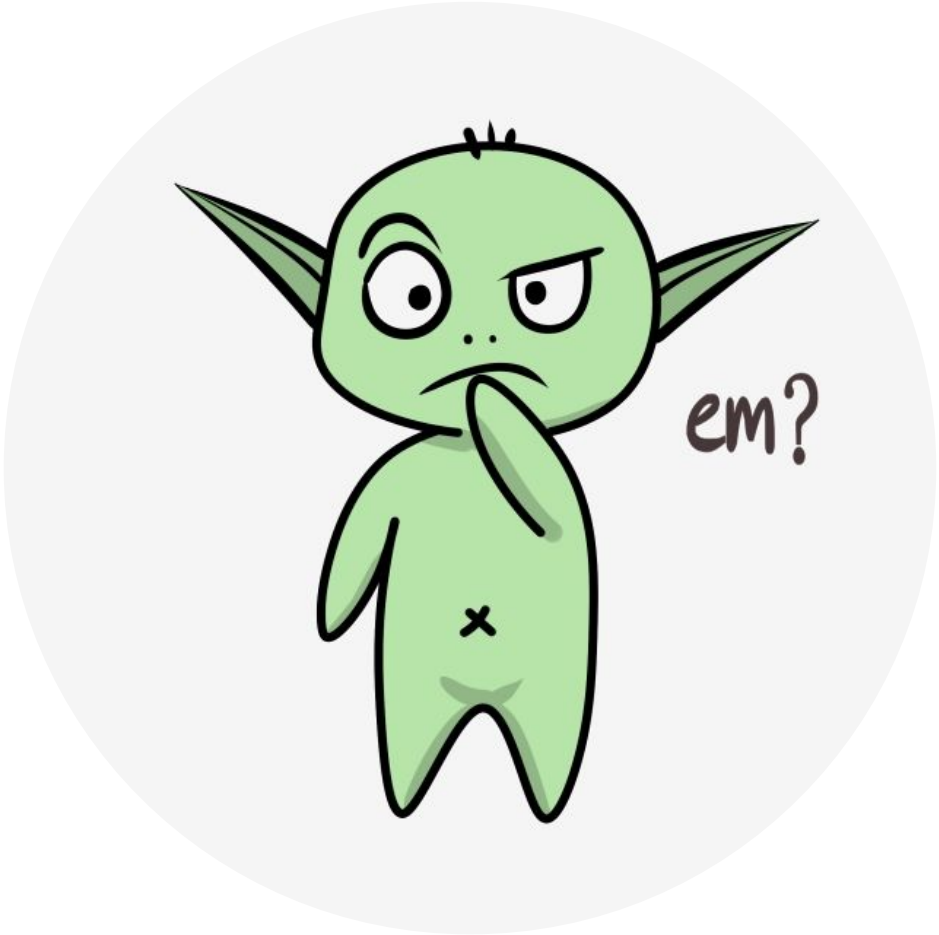
$R \leftarrow \text{bandit}(A)$

$N(A) \leftarrow N(A) + 1$

$Q(A) \leftarrow Q(A) + \frac{1}{N(A)} [R - Q(A)]$

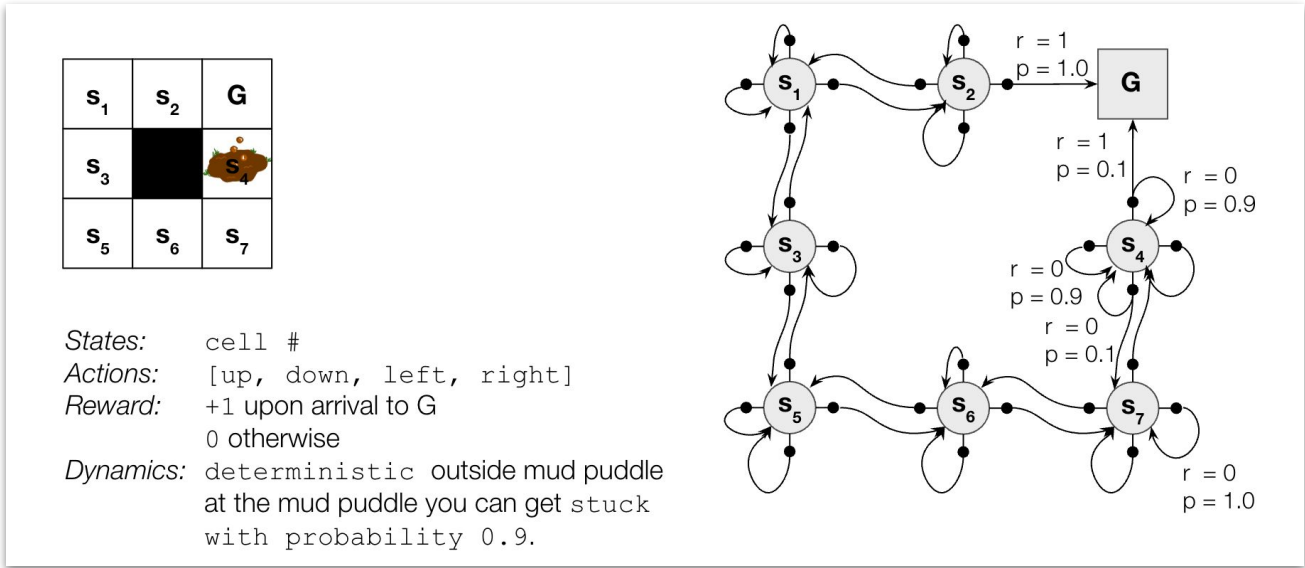
- Upper-confidence bound action selection

$$A_t \doteq \arg\max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$



Markov decision processes

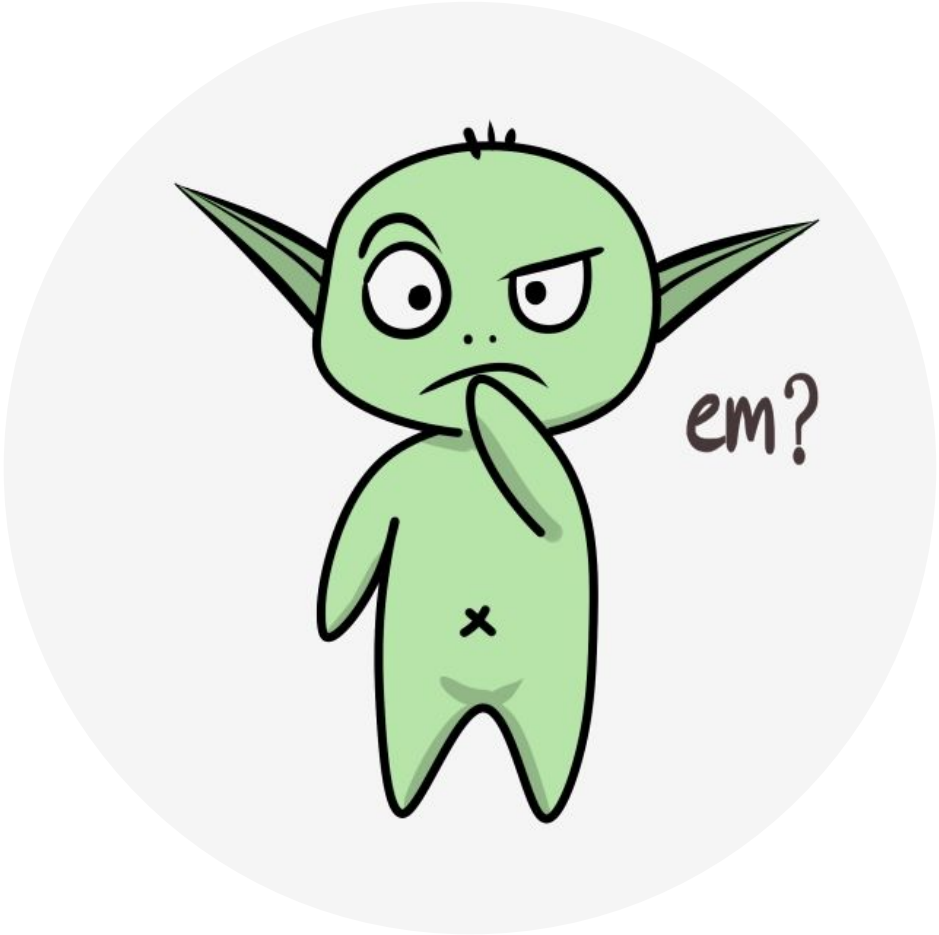
An MDP, $\mathcal{M}$, can be characterized as a tuple such that $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{R}, p, \gamma \rangle$. The agent's goal is to maximize the expected (discounted) return: $G_t \doteq \sum_{k=t+1}^T \gamma^{k-t-1} R_k$



Exercise 3.9 Suppose $\gamma = 0.9$ and the reward sequence is $R_1 = 2$ followed by an infinite sequence of 7s. What are G_1 and G_0 ? □

$$\begin{aligned} G_t &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \cdots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned} \tag{3.9}$$

$$G_t = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1-\gamma}. \tag{3.10}$$



Value functions and policies

- *Value functions are “functions of states (or state-action pairs) that estimate how good it is for the agent to be in a given state”.*
- “How good” means expected return.
- Expected returns depend on how the agent behaves, that is, its *policy*.
- A policy is a mapping from states to probabilities of selecting each possible action: $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$, in other words, $\pi(a|s)$ is the prob. that $A_t = a$ if $S_t = s$.

$$v_{\pi}(s) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = s] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s\right]$$

$$q_{\pi}(s, a) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a\right]$$

Value functions satisfy recursive relationships

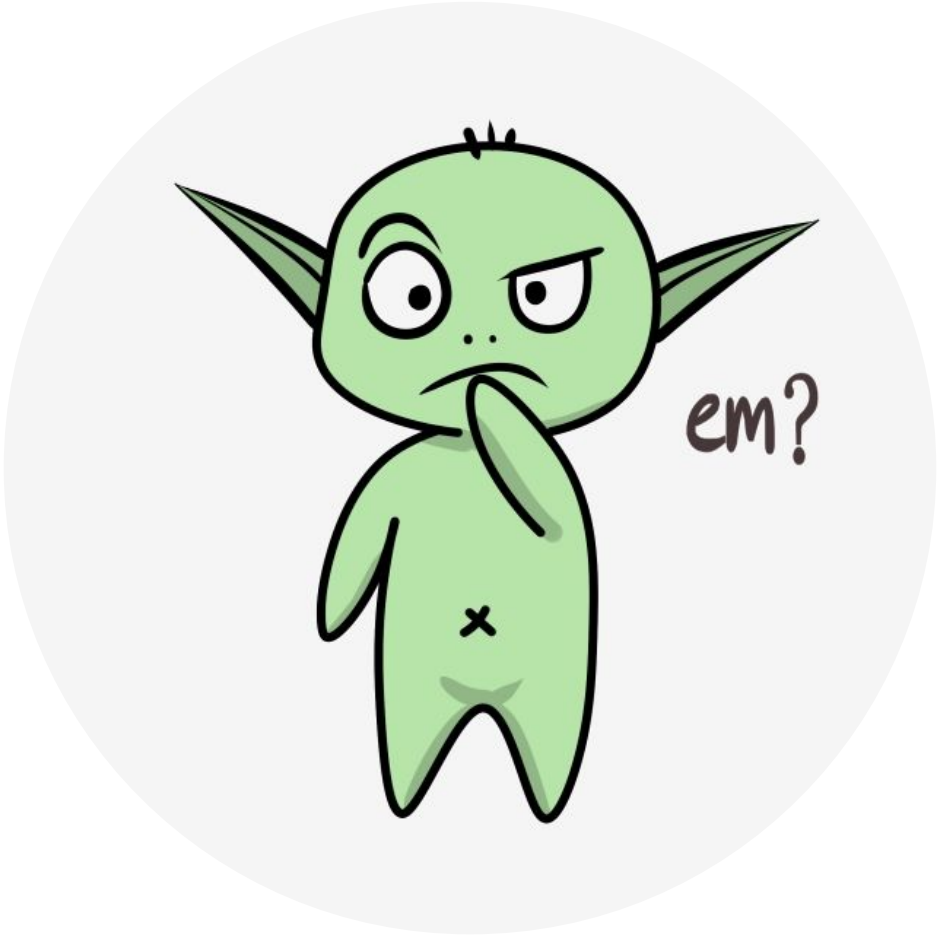
$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) \left[r + \gamma v_{\pi}(s') \right]$$

$$q_{\pi}(s, a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \sum_{a'} \pi(a'|s') q_{\pi}(s', a') \right]$$

$$v_*(s) = \max_a \sum_{s',r} p(s',r|s,a) \left[r + \gamma v_*(s') \right]$$

$$q_*(s, a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \max_{a'} q_*(s', a') \right]$$

Exercise: What are the backup diagrams for the four Bellman equations above?



Dynamic Programming

“To use value functions to organize and structure the search for good policies.”

*We use the same equations as before, but we replace $a =$ by $a \leftarrow$, that's it
(we turn Bellman equations into assignments).*

There's lots to decide

- What should we compute? $v_\pi, q_\pi, v_*, q_*, \pi^*$?
- How should we select states to imagine about? And in what order?
- How much computation do we need to figure out the optimal policy, π^* , using the function $p: \mathcal{S} \times \mathcal{R} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$?
- How many times do we need to iterate over this imagining / planning process?

Obtaining value functions and π^ from π and p (with no interaction) is called*
Dynamic Programming.

Policy Evaluation (Prediction)

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π , the policy to be evaluated

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$ arbitrarily, for $s \in \mathcal{S}$, and $V(\text{terminal})$ to 0

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

Policy Iteration: Interleaving Policy Eval. and Improvement

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(\text{terminal}) \doteq 0$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Value Iteration

Value Iteration, for estimating $\pi \approx \pi_*$

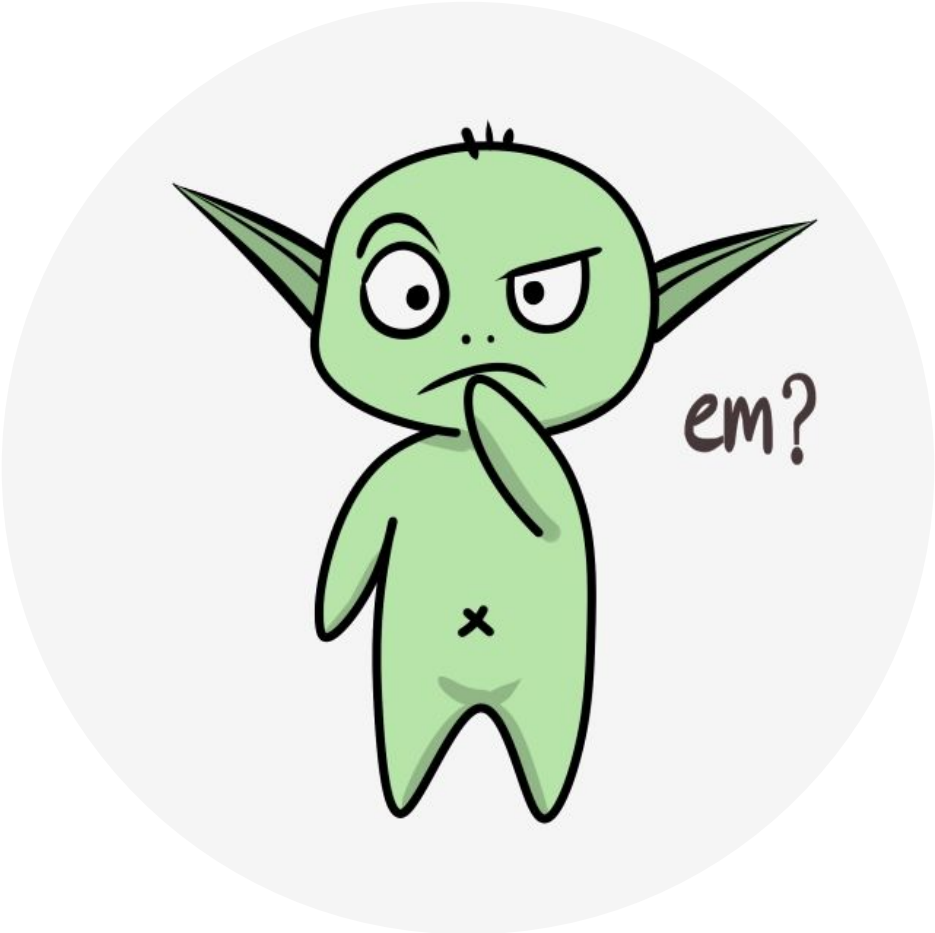
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
 Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

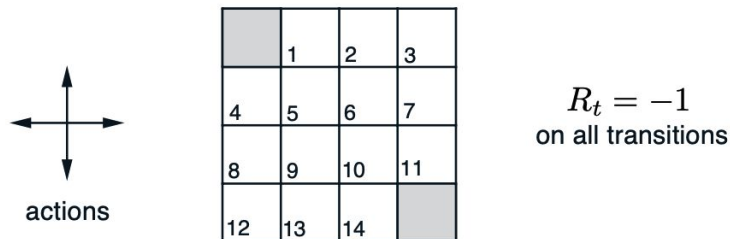
```
|  $\Delta \leftarrow 0$ 
| Loop for each  $s \in \mathcal{S}$ :
|    $v \leftarrow V(s)$ 
|    $V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$ 
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \operatorname{argmax}_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$



Example 4.1 Consider the 4×4 gridworld shown below.



The nonterminal states are $\mathcal{S} = \{1, 2, \dots, 14\}$. There are four actions possible in each state, $\mathcal{A} = \{\text{up}, \text{down}, \text{right}, \text{left}\}$, which deterministically cause the corresponding state transitions, except that actions that would take the agent off the grid in fact leave the state unchanged. Thus, for instance, $p(6, -1 | 5, \text{right}) = 1$, $p(7, -1 | 7, \text{right}) = 1$, and $p(10, r | 5, \text{right}) = 0$ for all $r \in \mathcal{R}$. This is an undiscounted, episodic task. The reward is -1 on all transitions until the terminal state is reached. The terminal state is shaded in the figure (although it is shown in two places, it is formally one state). The expected reward function is thus $r(s, a, s') = -1$ for all states s, s' and actions a . Suppose the agent follows the equiprobable random policy (all actions equally likely). The left side of Figure 4.1 shows the sequence of value functions $\{v_k\}$ computed by iterative policy evaluation. The final estimate is in fact v_π , which in this case gives for each state the negation of the expected number of steps from that state until termination. ■

Exercise 4.1 In Example 4.1, if π is the equiprobable random policy, what is $q_\pi(11, \text{down})$? What is $q_\pi(7, \text{down})$? □

Exercise 4.6 Suppose you are restricted to considering only policies that are ε -soft, meaning that the probability of selecting each action in each state, s , is at least $\varepsilon/|\mathcal{A}(s)|$. Describe qualitatively the changes that would be required in each of the steps 3, 2, and 1, in that order, of the policy iteration algorithm for v_* on page 80. $\square$

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(\text{terminal}) \doteq 0$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

